Machine Learning For Time Series Analysis and Forecasting

A Thesis Presented
by

Ming Luo

to

The Department of Mechanical & Industrial Engineering

in partial fulfillment of the requirements
for the degree of

Master of Science

in

Mechanical & Industrial Engineering

Northeastern University
Boston, Massachusetts

April 2023

To my family.

Contents

List of Figures iv

List of Tables v

Acknowledgments vi

Abstract of the Thesis vii

1 Introduction 1
- 1.1 Background . . . 1
- 1.2 What Are Time Series Challenges? . . . 2
- 1.3 Research Questions . . . 4
  - 1.3.1 Time Series Clustering and Classification . . . 4
  - 1.3.2 Hierarchical Time Series Forecasting . . . 7
- 1.4 Structure of Thesis . . . 8

2 Time Series Clustering and Classification 9
- 2.1 Literature Review . . . 9
- 2.2 Added-value Clustering and Classification . . . 10
  - 2.2.1 Framework of Added-value Clustering and Classification . . . 10
  - 2.2.2 Procedure to Added-values in Clustering . . . 14
- 2.3 Numerical Analysis . . . 15
  - 2.3.1 Set-up and Dataset . . . 15
  - 2.3.2 Evaluation Method . . . 16
  - 2.3.3 Clustering . . . 17
  - 2.3.4 Clustering Post-analysis . . . 17
  - 2.3.5 Actions After Clustering . . . 18
- 2.4 Summary . . . 22

3 Hierarchical Time Series Forecasting 24
- 3.1 Introduction . . . 24
- 3.2 Four Common Approaches . . . 28
  - 3.2.1 Bottom-up Approach . . . 28
  - 3.2.2 Top-down Approach . . . 31

3.2.3 Middle-out Approach 34
3.2.4 Optimal Reconciliation Approach 35
3.3 Framework of Reconciliation with Neural Network 35
3.4 Numerical Analysis 38
3.4.1 Goal of This Study 38
3.4.2 Dataset 39
3.4.3 Data Preprocessing 39
3.4.4 Model 39
3.4.5 Evaluation Method 42
3.4.6 Result 42
3.5 Summary 43

Bibliography **45**

List of Figures

2.1 Added-value Clustering and Classification Example 11
2.2 Clustering by added-values Procedures 12
2.3 Actions After Clustering Procedures 13
2.4 Training/Test Dataset 16
2.5 Clustering Result 17
2.6 Clustering A Graph 18
2.7 Clustering B Graph 19
2.8 Clustering C Graph 20
2.9 M3 accuracy Vs. M2 accuracy 21
2.10 Time Series CL4_34 Prediction Results 21
2.11 Time Series CL4_49 Prediction Results 22

3.1 A 2-level hierarchical tree diagram example 25
3.2 Reconciliation with neural network(R-NN) Framework 36
3.3 m-level Hierarchical Time Series (HTS) 37
3.4 Time Series Training and Testing Sets 40
3.5 R-NN Model Input and Ouput 40
3.6 R-NN Model Training Process 41
3.7 R-NN Model Prediction Process 42

List of Tables

3.1 Term Description of HTS Function 26
3.2 Term Description I 29
3.3 Term Description II 32
3.4 HTS Dataset Summary 39
3.5 Systematic Prediction Summary 43

Acknowledgments

Here I wish to express my most profound appreciation to my thesis advisor, Professor Paul Pei, for his wise guidance and kindness.

Abstract of the Thesis

Machine Learning For Time Series Analysis and Forecasting

by

Ming Luo

Master of Science in Data Analytics Engineering

Northeastern University, April 2023

Dr. Paul Pei, Advisor

We are immersed in a world with all types of data. Time series data are prevalent and essential in decision-making. Time series data have intrinsic temporal order and are thus immutable over time. The autocorrelation among time series data over time index makes them unique to deal with. Moreover, latent explanatory variables behind the time series make it challenging to handle.

In this thesis, the author applies machine learning techniques to analyze time series data for classification, clustering, and forecasting. First, a new distance measure, value-added, is proposed in time series classification and clustering. Further, the author develops a novel framework in which decisions such as the number of clusters and prediction based on value-added are made using different techniques. Numerical real-world data studies demonstrate the value-added framework in time series classification and clustering. Forecasting in scale is a particular issue in business forecasting. Frequent forecasting with a hierarchy of time series data differs entirely from that with a single univariate time series. The author first reviews standard hierarchical time series forecasting methods. Then a new approach reconciliation with neural network (R-NN) is proposed for hierarchical forecasting, considering the non-linear relationship among time series for forecasting. Conventional techniques have yet to incorporate the relationship between series while making individual predictions and tend to lose information. In addition, computational costs can be exceptionally high due to model perplexity when using methods such as optimal reconciliation. The R-NN is straightforward to implement and fast to train without losing information. Numerical studies are shown, and accuracy improvements are observed.

The methods and procedures developed in this thesis can be applied in various business settings. For example, the value-added time series classification and clustering procedure can be turned into a software product to classify and re-classify sales data for companies to order better and deliver. Business insights on clusters and predictions can be incorporated as well. Especially for

companies with limited labor resources to make predictions on plenty of products, this approach can still generate a good prognosis for all the series. Likewise, the reconciliation with neural network (R-NN) can be used to forecast many time series simultaneously consistently. Then a coherent hierarchy of forecasts can assist subsequent supply chain decisions such as production and deployment.

Chapter 1

Introduction

1.1 Background

Nowadays, we are immersed in a world where data is ubiquitous and essential. From national institutions to individuals, data has been collected in different ways. To understand them precisely, data is divided into categories and analyzed correspondingly. Time series and cross-sectional data are widespread and vital among all the data types. Cross-sectional data is collected at a given time, and its objects are multiple variables of interest. For example, a retail store manager gathered the sales of four kinds of ice creams last month. This cross-sectional dataset will have four numbers: the one-month total sales for each ice cream. In addition, the order of the cross-sectional data is flexible. The data can be represented in ascending, descending, or random order. However, time series data is quite different from cross-sectional data. Time series data has an intrinsic time order. This is because time series data is collected through a sequence of time points over regular intervals. One example is the daily sales of one kind of ice cream in July for the last three years. If we plot the data in a 2D graph, the x-axis will be the time point, and the y-axis will be the corresponding sales amount. Specifically, cross-sectional data is like taking a picture. The momentary information of all the participants matters. In contrast, time series data is like recording a video of a single participant over regular time intervals, like 9 am – 10 am every day. How the person reacts in the time order matters. All in all, what makes time series data unique is its intrinsic time order, which is that time series data is immutable in the time index.

With a given time series data, we can do either time series forecasting or time series analysis. The former aims at predicting future values over a period based on a given dataset accurately. The latter focuses on understanding the useful characteristics of the given dataset as much as possible

In daily life, we always hear the word forecasting when people mention future weather conditions. Forecasting can apply to various circumstances involving exerting historical data to predict future trends. For example, flower stores forecast the demand for flowers based on previous sales information. Forecasting is a vital tool in many fields since it can manage potential loss risk and prepare people more when dealing with uncertainties. Moreover, it elevates the efficiency of resource usage. Several forecasting models exist, such as the linear Regression model and artificial neural networks. One of the model-picking criteria is the nature of the given dataset. For instance, when the data shows repeating patterns along with the time, it indicates the data has a time series. One example is that the demand for roses peaks on February 24th every year on Valentine's Day. To forecast this kind of data, we need time series forecasting methods. Time series forecasting is a technique to predict future values over a period based on historical data with time order. Specifically, by assuming that the trends shown on the historical data will be similar to the future trends, time series forecasting estimates a sequence of future values by extrapolating necessary features from historical data such as trends, seasonal patterns, and irregular. Time series forecasting is widely used in many fields to help with decision-making, such as business, supply chain management, and production planning. In practice, we can utilize computer science techniques to analyze the time-related repeating change of the given data and then build mathematical models to predict future change.

1.2 What Are Time Series Challenges?

We already know that time series data are collected in chronological order. Therefore, the value of the observer at a one-time point is affected by the values of itself at a previous time point in one time series. We call this kind of dependency autocorrelation, also known as serial correlation. For example, a person's weight on different days of the week is autocorrelated. We should note that autocorrelation means the data is correlated with the observer, which is different from the correlation of two variables. Like correlation, the range of autocorrelation is also between -1 and 1. The absolute value of the autocorrelation indicates the intensity of the autocorrelation. When the value is closest to 1, it means the two values are heavily correlated. When it is close to 0, the two values are slightly correlated. The sign of autocorrelation indicates the direction of the autocorrelation. - sign means negative correlation, and + sign means positive correlation. For example, suppose a person's weight tends to rise tomorrow given that there is an increasing tendency of the past days' values. In that case, we will say that a person's weight exists in a positive autocorrelation. Conversely, if the person's weight tends to drop with the given increasing tendency, we will say that weight of a person

exists a negative autocorrelation. Moreover, the autocorrelation can also expand to the values in different intervals. First, let's assume that we collect a person's daily weight for eight weeks. We can measure the autocorrelation between a person's weight of today and the weight of the same person last Monday (assuming today is Monday). There is a time gap between the two observations, also known as lag. The time gap is a week. Therefore, there will be a lag seven autocorrelation if exists.

Now we are clear about the autocorrelation. Because of the autocorrelation, time series data is hard to handle. This is because most statistical models require the observations to be independent and random (the values of the observer are called observations). But the current value of the observer in the time series data depends on the previous values of itself. Let's take a closer look at why many statistical models don't apply to time series data. Take regression models as an example. If we want the model results trustworthy, the data set we implement must meet the underlying assumptions. Two main hypotheses are that the observations must be independent and errors (A.K.A. residuals) are independent and identically distributed (short for i.i.d). However, the time series data, which has an inherited time order, are autocorrelated. The past values affect the current value. Therefore, the observations are dependent. In addition, we use the residual to measure the rightness of the generated model. The residual is the difference between the predicted and actual observations at the same given time point. Since real observations are autocorrelated, the residuals that contain them are also autocorrelated. Therefore, we miss the required assumptions and can't use the model. The time series data is hard to deal with since its natural time order and the autocorrelation between its observations.

In addition to autocorrelation, there are other obstacles when dealing with time series data: within-category correlation and between-category correlation. Let's talk about them one by one. Within-category correlation measures the extent of one item's influence on another when both belong to the same class. Here is an example to illustrate within-category correlation. Given that two items under the drink category, orange juice and water, are offered if they have a negative within-category correlation relationship. It means that more orange juice will lead to less water consumption. It makes sense since people have limitations on the total amount of liquid they can take in a fixed period. Besides within-category correlation, between-category correlation (A.K.A. cross-category correlation) is also pervasive in time series data. Between-category correlation tests the potential relationship embedded in items from various classes. This kind of correlation is commonly used in business to predict the multi-items in the final shopping basket instead of a single thing. In short, between-category correlation is about the likelihood of the appearance of a bundle. Since there are usually multi-category items in the consumer's shopping basket, people are curious whether things across different categories are interdependent. This is the crucial distinct difference between within-

category correlation and between-category correlation. The former cares about the interdependency between items in a single class. One example of a positive between-category correlation is that people who buy the air fryer, a kind of kitchen appliance, also put frozen French fries, a type of food, in the shopping cart.

Moreover, dealing with time series data can be daunting due to its large scale. Generally speaking, to better understand the topic, the size of related data tends to increase dramatically. For example, if basic patterns are missed, researchers collect time series data in the minor possible time interval. For example, to find the temperature change pattern in the past seven days, the scientists will contain hourly temperature values instead of daily temperature values. The former increases the data size from 7 to 128, 23 times greater than the latter. With the vast number of time series, the computational complexity increases significantly due to the heavy task of modeling each time series. Not to mention the improved space complexity results from the big data size. In addition, as the data volume increases, the data tends to be noisier. And the correct time series patterns get harder to catch, which will inevitably end up creating some inaccurate forecasting or classification models.

Besides all the quantitative correlations, time series is also related to explanatory factors. The explanatory factors, such as festivals and promotion sales, can be analyzed qualitatively. For instance, when the electronics retailer plots the monthly sales for the last three years, the company might see a bump in November on the graph. That is because of the Black Friday Deal. Therefore, to generate an accurate model, scientists should consider measurable and explanatory factors. This extends the time series data difficulty level.

Above are the main challenges of time series data. Although it is hard to deal with, scientists have found multiple methods to resolve the challenges.

1.3 Research Questions

1.3.1 Time Series Clustering and Classification

Adapting Machine learning techniques to time series forecasting has been popular recently. There are two machine learning approaches to group a set of time series and then analyze each group. One is related to an unsupervised learning technique called clustering, and the other one is a supervised learning technique called classification. Although both clustering and classification are commonly applied to cross-sectional data, we adapt them so that time series data can also be analyzed with these approaches.

First, I am going to introduce some background knowledge about clustering. Clustering separates all the objects into different groups based on their similarities. Each of these groups is called one cluster. Suppose we plot all the observation records in a 2-dimensional graph. In that case, the x-axis represents the observers, and the y-axis represents the value of the observers (observation). We can see that the members in the same cluster are close to each other, and that cluster is relatively far from other clusters. One of the clustering goals is to minimize the distance within one cluster and maximize the distance between clusters. We can also use clustering in the data preprocessing step to reduce the data dimensions. One example of clustering is supermarkets separating their items into different groups, such as bakery, meat, and dairy. Cupcakes, apple pies, and cookies are all under the bakery cluster. With clustering, the store can make business plans efficiently based on the analysis with clusters. This is because the decision-maker can see a general trend from the cluster and not get bothered by the noise from individual items. And the decision-maker can make individual forecasts with the best-fitting model for each of the groups to make better accurate predictions. One thing that needs to be noticed is that data clustering is spontaneous. The intrinsic similarity between the data makes them cluster together naturally.

In addition, scientists divide clustering into two ways: hierarchical clustering and non-hierarchical clustering. Hierarchical clustering needs to follow the nested tree-based structure to group data. The clustering process should either be root to stem to leaf or in the opposite direction. Hierarchical clustering can also be split into two methods, agglomerative and divisive clustering, which are opposite. Agglomerative clustering, a bottom-up approach, repeats the clustering method on the generated clusters from the previous stage until a single giant cluster is formed. Divisive clustering, a top-down method, is repeated to separate one big cluster, and eventually, many clusters are formed. Although these two hierarchical clustering methods seem different, they keep the data under the hierarchical structure. However, non-hierarchical clustering does not require data to have hierarchical order. It partitions data into other clusters directly without considering tree-like systems. Compared to hierarchical clustering, non-hierarchical clustering has some advantages: it is relatively stable and computationally faster. Yet, its disadvantage is that it is hard to decide the number of clusters.

Then, it is time for classification. Classification uses input variables to generate a model to separate the output variable into several categories. For instance, in a given dataset, we have input variables: blood pressure, heart rate, body mass index (BMI), and output variable, the body condition score. We can generate a model to classify people into three categories: healthy condition, average condition, and bad condition. With this model, we can detect a new person's health condition with

only the input information, values of blood pressure, heart rate, and BMI. With classification, we can see the interval relationship between items and predict the new item to generate a fast and accurate initial understanding.

As we mentioned above, both clustering and classification are for cross-sectional data. One of the main differences between them is that classification requires labeled data, while clustering does not need the data to have labels. In addition, cross-sectional data does not have time order and is static, while time series data does and is dynamic. To apply clustering and classification on time series data, we need to use techniques to extract time series features and apply different methods to measure similarity. However, if labeling all the data is unavailable or too expensive, we can use semi-supervised clustering or classification instead, aiming to achieve high-quality clustering or classification with less time and costs.

Let's talk about time series clustering first. We primarily use hierarchical clustering to cluster time series. Instead of finding the similarity of data points, we measure the proximity of the dynamic feature of the time series and group the similar time series into different clusters. For example, we have 10-time series on hand, and with time series clustering, we can group them into three clusters. And then analyze the three groups. Furthermore, one of the hierarchical clustering methods, agglomerative clustering, is commonly used to cluster time series.

Next is time series classification. The procedure of the time series classification is like the rest of the classification. We give each time series a label using the output variable. Then we divide the time series into three groups: training, validation, and testing. We train the time series classification models on the training dataset, use the validation dataset to pick the best model, and test the picked model to check its quality. However, the distinction of time series classification is that we need to keep the natural time order of the data. Within the time series classification model, when a new time series is introduced, its pattern will be compared with all the existing classes and assigned to the most similar one. Then the new time series will be labeled as that class.

Due to the many similarity measurements, such as Euclidean distance, ignoring the auto-correlation problem which commonly appears in time series data and causes high computational complexity, we use other techniques to measure the similarity between time series, such as Manhattan distance or Mahalanobis distance. In addition, time series data are high-dimensional datasets. We need to reduce the dataset dimensions to do similarity measurements. We can measure the similarity of each time series by computing the distance of different measurements, such as the mean or standard deviation of each time series. We can also apply the Pareto principle (the 80/20 rule) to cluster data. The Pareto principle means that roughly 80 percent of the outcome results from 20 percent of the

input information. Identifying the priority ahead provides a cost and time-efficient way to impact significantly.

In this thesis, we address the question of how we can measure the similarity between two time series accurately and cost-efficiently. Furthermore, we can use generated clusters to optimize prediction accuracy with machine learning and deep learning.

1.3.2 Hierarchical Time Series Forecasting

Hierarchical Time Series (HTS) is a set of time series that contains a hierarchical aggregation structure. For example, three coffee shops are owned by the same company in the US. Two of them are located in Boston, and the other one is located in New York City. For each store, the daily coffee drinks sales are a time series. The sum of daily coffee drink sales in each city is also a time series. Two coffee stores aggregate the deals in Boston. In addition, the overall daily coffee drinks sales for all three stores are also a time series, which is a sum on the country level. All these 6-time series (3 at the store level, two at the city level, and one at the country level) construct a hierarchy. HTS has a tree-like structure with roots, stems, and leaves. In the above example, the root is the time series at the country level, the stem is the time series at the city level, and the leaves are the time series at the store level. In the HTS, if we want to trace from the root to the leaf, we must strictly follow the tree structure.

HTS forecasting is a set of forecasting methods consistent across various levels in the hierarchy structure. This is because all the time series are related and connected in sequence. In addition, the HTS forecasting time scale varies based on the requirements. There are long-term forecasting and short-term forecasting. For example, a small bakery store owner may need daily sales forecasting, a short-term one, so that they can prepare the right amount of fresh food. A big furniture company may need yearly sales forecasting to make a long-term plan to help them prepare the materials and produce the items. Although long-term and short-term forecasting is not unique traits for HTS, long-term forecasting and short-term HTS forecasting can be used for each other by disaggregation or aggregation. For example, yearly HTS forecasting can be done by aggregating the monthly HTS forecasting. In addition, some special events, like holidays, will also require long-term HTS forecasting. This is because many holidays, such as Thanksgiving Day and Christmas, happen once a year. With the yearly HTS forecasting, we will find the patterns that repeat once a year.

Besides, HTS forecasting at different time scales can also be generated at different dimensions. For example, given the daily sales of a bakery shop, we will have one-dimensional data, the

value of sales, which changes along with time. We can use the univariate time series generated from this one-dimensional data to forecast daily sales for this bakery shop. What's more, if we can get this bakery store's daily sales and volume, we will have two variables, or two dimensions, which vary along with time. We can use the multivariate time series generated from two-dimensional data to forecast daily sales for this bakery shop. In addition, if we get a dataset containing daily sales of ten unrelated bakery shops, this dataset includes a 10-time series, whose values are collected individually. This is a multiple-time series.

In this thesis, we further address the questions of how we can explore time series dependencies in the hierarchical structure and forecast at scale. Furthermore, how machine learning and deep learning can contribute to hierarchical forecast.

1.4 Structure of Thesis

In the following chapters, we will introduce time series clustering and classification in section 2 and hierarchical time series analysis in section 3. In both sections, we organized in the same logic: first, introduce current methods and their advantages and disadvantages. Second, we propose our new methods and procedures. Further, we highlight our contributions. Then, we will do numerical analysis to show the efficiency of our methods. Last, end the section with summary.

In addition, the dataset we use for section 2 and section 3 numerical analysis derives from the same original data. The original data is about sales orders of 7 lines of business, 400 products across 1007 customers in a top beverage company over more than two years, spanning from January 2015 to February 2017.

Chapter 2

Time Series Clustering and Classification

2.1 Literature Review

In clustering, a model's two most critical determining factors are the clustering algorithms and the distance measurement methods. There are different clustering algorithms categories, such as partitional, density-based, and hierarchical categories. A viral algorithm, the k-means clustering algorithm, is under the partitional category. Another popular algorithm, the agglomerative clustering algorithm, belongs to the hierarchical category. It treats each time series as a cluster, then merges the two similar clusters into a bigger one. Repeat the process until all the clusters become one [1].

In addition, picking the proper distance measurement, such as Euclidean distance and dynamic time warping (DTW), is critical to the clustering performance. Euclidean distance is the most popular distance measurement method, but it has limitations in measuring time series data due to its sensitivity to time shifts. A more accurate measurement method is dynamic time warping (DWT). DTW is to minimize the sum of lines. The procedure to get the lines is to connect two time series with multiple lines that don't cross. And each end of the line is a point on either one of the two time series. This way, we can still detect two similar time series, even one shift subtly [1].

In classification, There are many algorithms such as distance-based approaches, shapelet-based approaches, Interval-based approaches, and dictionary-based classification. They aim at predicting the label of new time series with a model that is trained with a set of labeled time series. Among all the algorithms, a distance-based approach, k-nearest neighbors (k-nn) is very popular. A widely used benchmark classification method is to use DTW to calculate distance and then use 1-NN to decide which label the new data should assign to [2].

Time series clustering and classification can be used in many areas such as recognizing

dynamic changes in time series, prediction and recommendation, and pattern discovery [3]. In this paper, we use time series clustering and classification in the preprocessing step to group time series and generate predictions for each cluster to support better decision-making. Among all kinds of clustering and classification algorithms, we are interested in finding the best algorithm for each question. Therefore, we address the validation for time series clustering and classification.

There are two main branches in evaluation measurement techniques: visualization and scalar accuracy measure. The scalar accuracy measurements include external indices and internal indices, which use real numbers to indicate the accuracy of cluster methods [3]. External indices can be applied when the true class labels for each data point are accessible while internal indices don't need class labels [1]. External indices such as the rand index, cluster purity, and entropy measure the closeness of the formed clusters, which are generated from clustering methods to the true class labels, which are given by human experts and considered as the perfect clustering. However, although external indices are more popular than internal indices, we should apply internal indices when the class labels are not available. Internal indices such as the R-squared index and sum of squared error are objective functions that aim at high similarity within one cluster and high dissimilarity between clusters. The sum of squared error is the most popular internal index, which uses a lower value to represent better clustering. In addition, the error is the distance between one cluster and its closest cluster [3].

2.2 Added-value Clustering and Classification

2.2.1 Framework of Added-value Clustering and Classification

Now we introduce a new way to do clustering and classification called added-value clustering and classification. I illustrate how added-value clustering and classification work with the following example. For instance, if we have three time series corresponding to three different items in a retail store: beverage I, II, and III. Ideally, it will be perfect for predicting every time series accurately. However, analyzing and predicting each time series will take lots of effort and money. In reality, the retail store only has limited resources for its items. Therefore, the retail store should focus on forecasting some things whose prediction accuracy can be improved by exploring different prediction algorithms and reducing priority for other items whose prediction accuracy with varying prediction approaches remains or is even worse than that with naive methods.

Here we apply clustering and classification to group items into different categories by

following rules. One of the clustering criteria, which we call added-value, could be the prediction accuracy rate improved by applying a state-of-the-art prediction algorithm to the time series, compared with applying the naive prediction model. The reason we call it added-value is that it is the margin of improvement in prediction accuracy from the time series analysis.

The figure 2.1 is a framework of how to use added-value clustering and classification with three time series. First, we apply the same naive model such as moving average to the three time series, and get prediction accuracy values for each of them: 65%, 55%, and 60%. Second, we apply the same alternative methods such as AutoRegressive Integrated Moving Average (ARIMA) model to each of the time series and also get prediction accuracy values: 80%, 75%, and 50%. Next, we calculate the added-value for each of the time series based on the equation: added-value = prediction accuracy of alternative method - prediction accuracy of the naive method. For instance, the added-value for time series I is 80% - 65% = 15%. With the same procedure, we get the added-values for time series II and III, which are +20% and -10%. Then, we decide the number of clusters, which is 2, and the range of added-values in each cluster, which are $(-\infty, 0], (0, +\infty)$ in this case. Time series I and II belong to cluster A and time series III is in cluster B. After clustering time series, we decide which clusters are worth the effort to make further optimization. In our case, we choose to further improve the prediction accuracy for time series in cluster A since the alternative approach works better than the naive method and we could make further improvements by applying some other method.

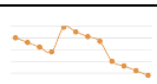
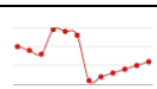
Cluster	Time Series	Graph	Prediction Accuracy		Added Value
			Naive Method	Alternative Method	
A	I		65%	80%	+15%
	II		55%	75%	+20%
B	III		60%	50%	-10%

Figure 2.1: Added-value Clustering and Classification Example

Let's move from the specific example to the general procedures of time series clustering and classification. Below are two flowcharts to illustrate the complete process of time series clustering

and classification: the first flowchart, which is named Part I. Flowchart of clustering and classification, shows how to cluster and classify the raw time series data into several categories, or clusters, see figure 2.2. The second flowchart is named Part 2. The flowchart of actions after clustering and classification indicates how to take advantage of the clustered time series, see figure 2.3.

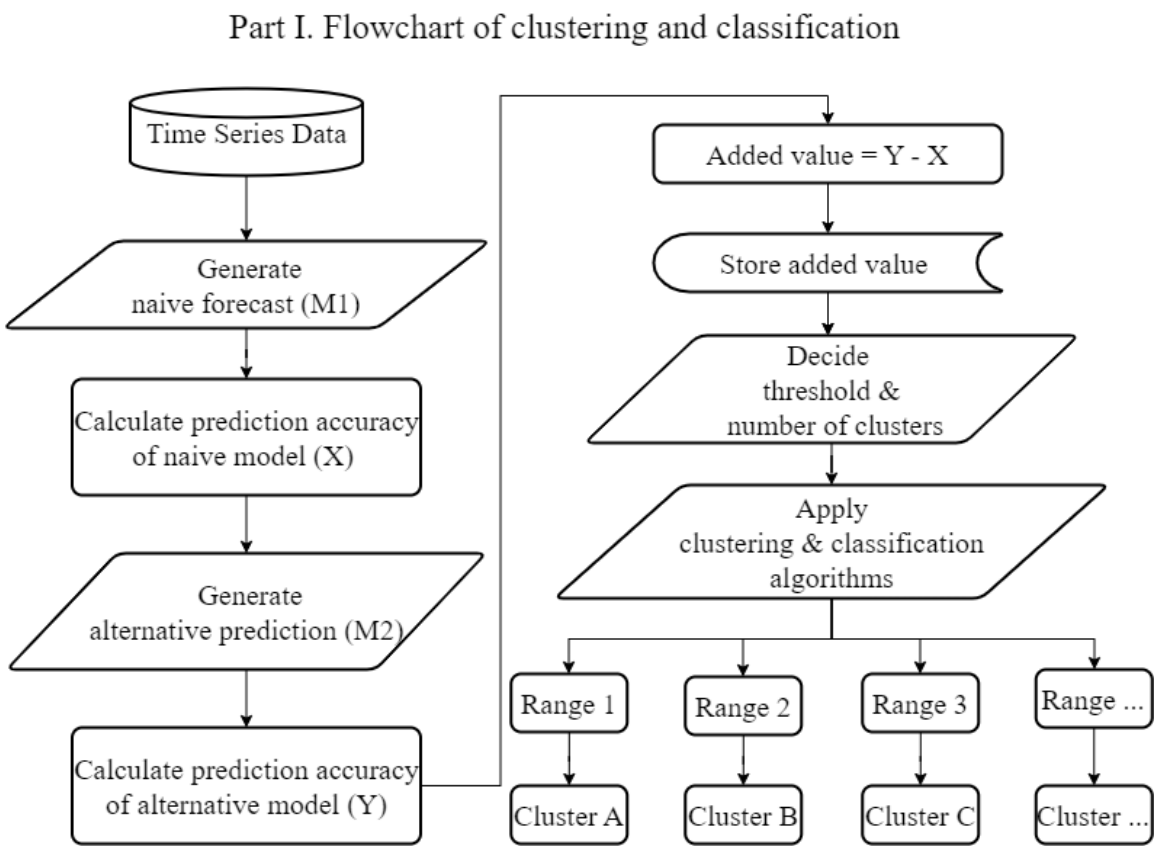


Figure 2.2: Clustering by Added-value Procedure

In the clustering and classification process, we first input the time series data and then generate naive forecasting for each time series. Here we denote the naive forecasting model M1. Then calculate the prediction accuracy of the naive model for each time series. Here we denote it X. Similarly, we generate an alternative prediction model, M2. Then apply the same time series data to it and get the prediction accuracy of it for each time series, which is named Y. For each time series, we calculate its added-value by subtracting the prediction accuracy of the naive model from the prediction accuracy of the alternative prediction model, which is (Y - X). Next, we store all the added-values for future use. With all the added-values, the searchers can decide the number of clusters and

thresholds based on past experiences, goals, or domain knowledge. In the end, we apply clustering and classification algorithms to group all the time series. We can apply several clustering and classification algorithms and pick one of the best after applying evaluation measurement techniques.

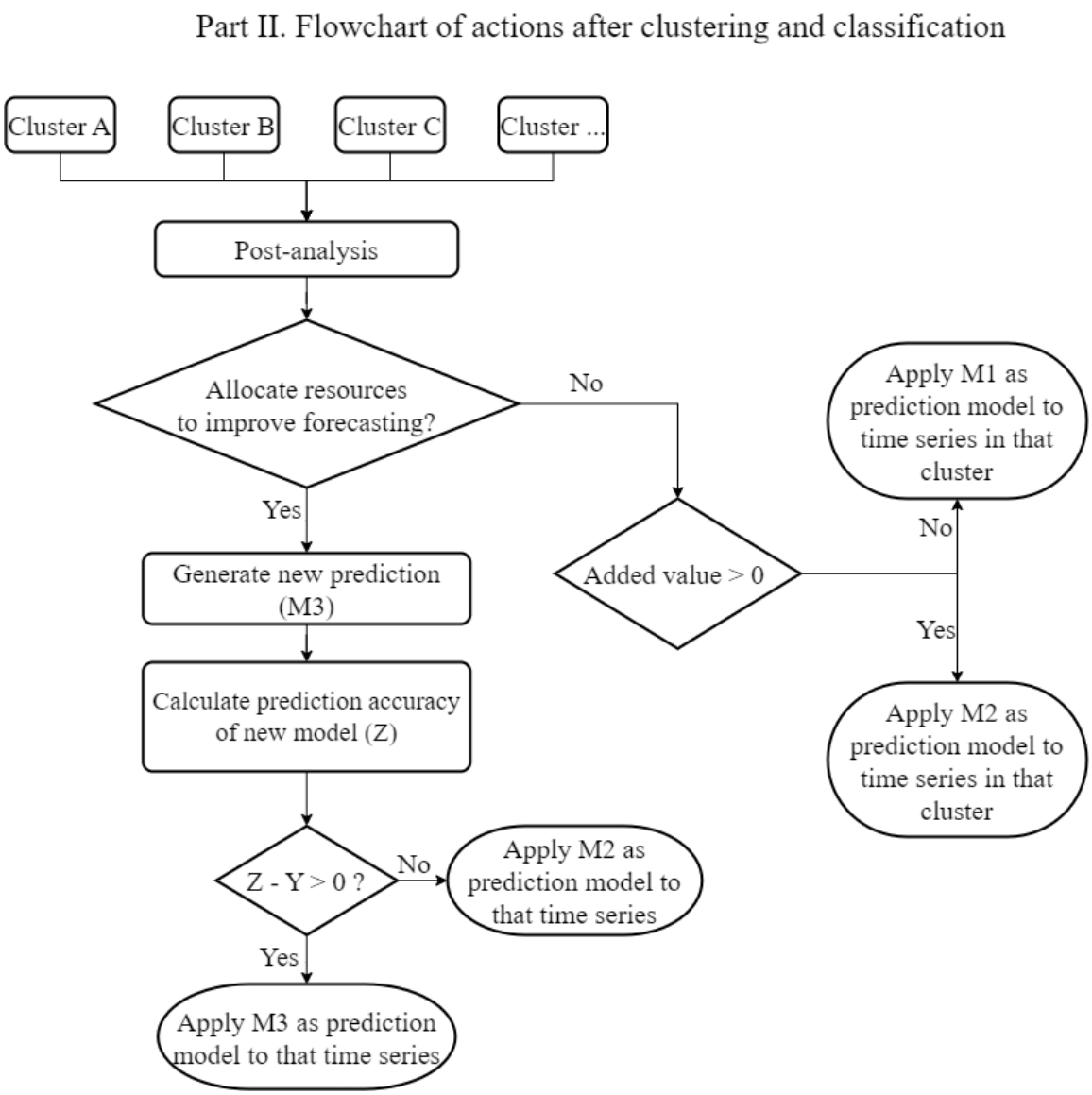


Figure 2.3: Actions After Clustering Procedure

After clustering and classification, we decide what to do with each cluster. The cluster action is affected by the project's purpose and available resources.

We first analyze these clusters, such as the number of time series in each cluster and the

percentage of the time series in each cluster over the total number of time series in the raw dataset. Then according to the goal and the available resources, we take action for each cluster. For instance, if a company has very limited labor hours to distribute on improving forecasting accuracy, it could focus on improving the time series forecasting accuracy of cluster A in which added-values of included time series are the highest compared to that in other clusters. In this case, we will generate a new prediction model, M3, to apply to every time series in cluster A. If the prediction accuracy of M3, Z, is higher than the prediction accuracy of M2 for one time series, then we apply the new prediction model, M3, to that time series. Otherwise, we will use the alternative prediction model, M2, to predict the future event of that time series. For clusters except for cluster A, we will apply M1 or M2 based on the added-values of that cluster. If most of the added-values in one cluster are less than 0, the prediction accuracy of the alternative prediction model, M2, performs worse than the naive model, M1. Therefore, we will apply M1 to the time series in that cluster. Similarly, if most of the added-values in one cluster are more than 0, we will use M2 for the time series in that cluster since M2 performs better than M1. The objective is to prioritize and allocate resources over customers, which has the highest added-values when the resources are limited. If more resources are available, we can generate M3 for more clusters as we did for cluster A.

2.2.2 Procedure to Added-values in Clustering

When choosing the proper algorithms to do clustering and classification, one should consider its business goal, the dataset's nature, and the different algorithms' capability. Several algorithms are available such as hierarchical clustering, centroid-based, and density-based clustering. They are prominent in different ways. Hierarchical clustering could give a clear structure of clusters and could cluster fast when the time series data is at scale. Centroid-based clustering is easy to implement. And density-based clustering is good at handling noisy datasets.

In the time series clustering and classification process, we generate three models: the naive prediction model (M1), the alternative prediction model(M2), and the new prediction model(M3). The M1 and M2 will apply to every time series in the raw dataset to get an added-value for each. Then we use the added-value to group time series into different clusters. Once we have different clusters, we will choose how to allocate resources to other clusters. For the preferred clusters, which contain large added-values, we will apply M3 to them to further improve the forecasting accuracy. For the non-preferred clusters, we will not further improve their forecasting accuracy.

The reason why we need M1 is that we need a benchmark that represents the performance

of conventional models. The M1 models are the same across all time series as benchmark forecasts. The reason why we need M2 is that we need to see if the prediction accuracy of one time series can be improved by applying a more advanced model. The difference between the prediction accuracy of M1 and M2 is called added-value to cluster time series. The added-value represents the performance advantages in accuracy by using M2 over M1. If the added-value of a time series is positive, it means that the prediction accuracy of that time series can be improved by applying an advanced model. Therefore, we need M3 to improve the performance in accuracy further.

2.3 Numerical Analysis

2.3.1 Set-up and Dataset

This study aims to cluster the time series into groups with added-values and then forecast the orders for the next six months for each time series by applying different approaches. The forecasting methods being used in this study are naive forecasting, dynamic forecasting, and multilayer perceptron model-based forecasting (MLP).

We use two datasets for this numerical analysis. One describes the monthly order information from January 2015 to February 2017 in 51 regions. The value for each time series in this dataset is the monthly order amount in one area. The other one is the prediction of monthly orders with the ARIMA model from January 2015 to February 2017 in 51 regions. We will use this dataset as the M2 prediction result to calculate the added-values later.

As introduced earlier, time series data is a sequence of time-ordered data, which doesn't have inputs and outputs. In order to apply the data to the machine learning models that requires input and output values for training, we need to transfer the data into a supervised learning data [4]. As we know that the time series value is highly correlated with each other, one value in the sequence is influenced by the previous values and will influence the following values. In our dataset, we set the first value of input variable is the list with first 6 values of original time series sequence and the value of output variable is the 7th value. Then the second input value is the list with 2th to 7th values and the value of output variable is the 8th value. Repeating this procedure until the last value in the sequence become output values. We have 26 values in each time series, since the first 5 values can not be the output value, there is 20 inputs and 20 outputs. After we re-organize all the dataset, we will split the dataset into training and test dataset. We are interested in predicting the the monthly sales from 2016 September to 2017 February, which is the last 6 outputs. We use the first 14 inputs

and outputs as the training set, and the last 6 values as the testing set. Below is the graph of the time series CL4_15 training and test dataset, see Figure 2.4:

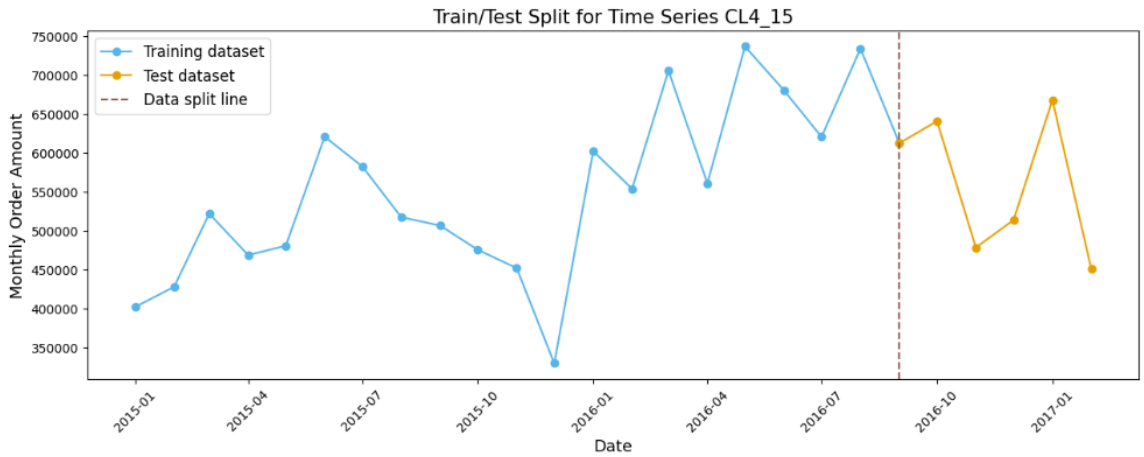


Figure 2.4: Training/Test Dataset

The naive prediction model (M1) in this study is the naive method, which is set as the baseline. In the naive method, we assume that the prediction value for a given time, T, is the actual value of the time, $T-1$.

We use the forecasting result in the given dataset as the alternative prediction model, the M2 prediction result. The given prediction results are from the ARIMA model made by the company's experts and market insights incorporated. Performance has been proven relatively good.

The new prediction model(M3) being used in this study is a multilayer perceptron(MLP) model. The MLP model has one input layer with six nodes, one hidden layer of 100 nodes, and one output layer with one node with Adam as the optimizer, ReLu as the activation function, and mean squared error as its loss function.

2.3.2 Evaluation Method

We use accuracy to compare performance under different approaches for each time series. We use the values of monthly orders from January 2015 to August 2016 to train the models and then get the prediction values of monthly orders from September 2016 to February 2017.

$$Accuracy = 1 - Error$$
$$Error = \frac{Sum|\text{(actual value) - (prediction value)}|}{Sum(\text{actual value})}$$

Please note, when both the numerator, $Sum|$(actual value) - (prediction value)$|$, and denominator, Sum (actual values) is 0, we set the accuracy as 1. When the numerator is not 0, but denominator is 0, we set the accuracy as 0.

2.3.3 Clustering

We first apply the Naive Method to each of the time series and get the accuracy value, Acc_m1. Then compare the forecast order values and actual order value to calculate the accuracy value for M2, Acc_m2. Then subtract Acc_m1 from Acc_m2 to generate the added-values for each time series. If the added-value is positive, it means that the M2 model performs better than the naive model. The higher the positive added-value is, the better the M2 model performs. Conversely, if the added-value is negative, it means that the M2 model performs worse than the naive model. Therefore, we can cluster the time series into three groups based on the added-values:

- Cluster A, contains some time series with the added-value of no more than 0.
- Cluster B, contains some time series with added-value more than 0 but no more than 0.1.
- Cluster C, contains some time series with added-value more than 0.1.

2.3.4 Clustering Post-analysis

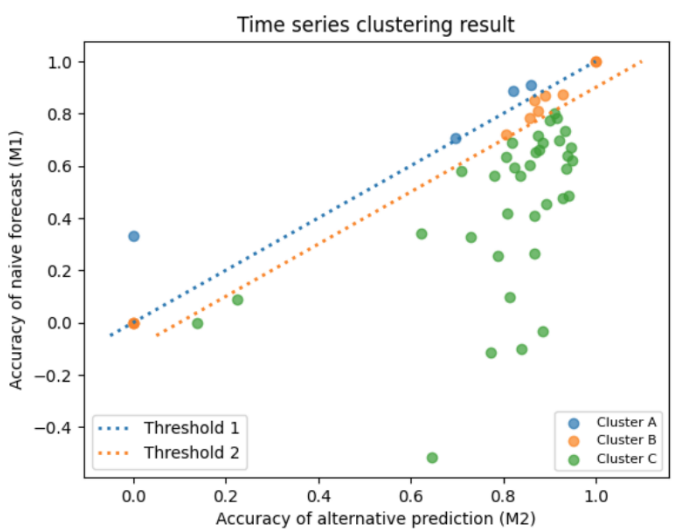


Figure 2.5: Clustering Result

Figure 2.5 is the plot of time series clustering result. 7.8% of the time series is in Cluster A, which has a total number of 4 time series. The blue dots in the figure 2.5 represent the time series in cluster A. 21.6% of the time series is in Cluster B, which has a total number of 11 time series. The orange dots in the figure 2.5 represent the time series in cluster A. And the rest 70.6% of the time series is in Cluster C, which has a total number of 36 time series. The green dots in the figure 2.5 represent the time series in cluster C. In addition, blue dash line represents threshold 0 and Orange line represents threshold 0.1. Below are the time series charts for each cluster, see figure 2.6, figure 2.7, and figure 2.8.

In cluster A, see figure 2.6, which contains 4 time series whose added-value is less than 0, we can observe that values for most of the time series are small. And even though the time series, CL4_29 has relatively large values, but its values drop dramatically.

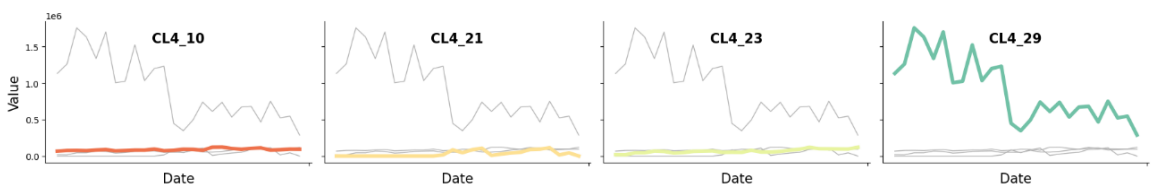


Figure 2.6: Clustering A Graph

In cluster B, see figure 2.7, which contains 11 time series whose added-value is between 0 and 0.1, We can observe that for this cluster, advanced ARIMA models add more values than the naive model. Further, it matters more to the business as we observe a larger number of time series in this cluster.

In cluster C, see figure 2.8, which contains 36 time series whose added-value is more than 0.1, We further observe more time series with higher added-values in cluster C. Moreover, it means that a large number of time series has achieved descent performance with ARIMA forecasting models used in M2. This provides a solid foundation in time series classification and clustering.

2.3.5 Actions After Clustering

Since we have limited resources, such as labor hours, to work on the prediction, we would like to allocate time to cluster B to improve the prediction performance. We see the promise that the advanced model can increase its accuracy from the previous comparison. This is because the time series in cluster A is hard to predict, and applying M2 to the time series in cluster C already generates a good prediction. Therefore, the cost of improving time series prediction performance in

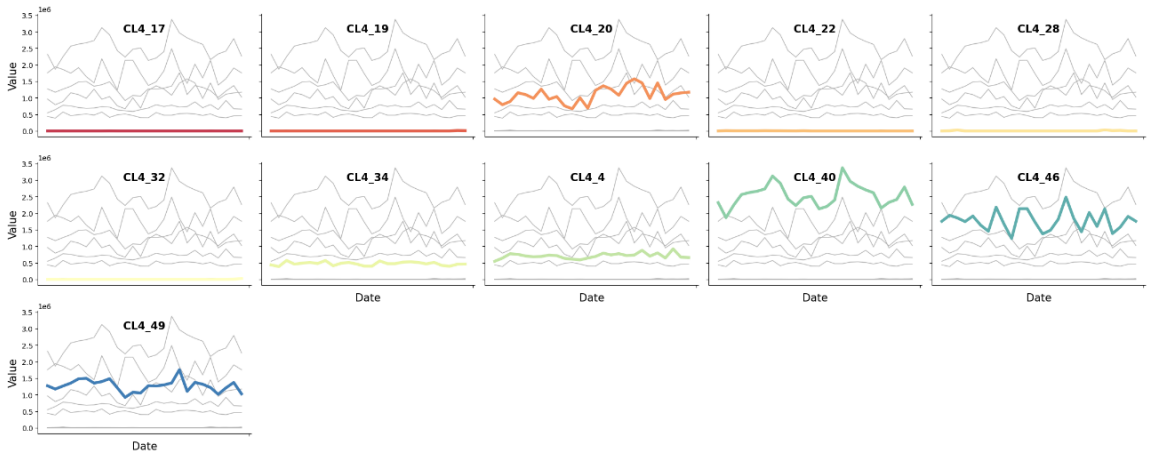


Figure 2.7: Clustering B Graph

cluster B by 1%, which is the marginal cost, is the lowest compared to that in the other two clusters. Therefore, we further generate a third model to improve the prediction accuracy of the time series in cluster B. The time series in cluster A keep using the predicted result from the naive method, and the time series in cluster C keep using the expected outcome from M2. We will focus on the time series in cluster B.

Since we have a relatively small dataset, to avoid overfitting, we applied a simple MLP model to the time series in cluster B. Below is the result of comparing the M3 accuracy and M2 accuracy, see figure 2.9.

We can see that three time series, CL4_28, CL4_34, and CL4_49, prediction accuracy gets improved. According to the frameworks we mentioned before, we will apply the M3 model to these three time series. The time series CL4_19 and CL4_32 prediction accuracy remains, we will apply either M2 or M3 to them. The remaining six time series prediction accuracy get decreased, so we still apply m2 model to them.

In addition, we plot the graph of prediction results of time series CL4_34 and CL4_49 with three methods to gain some insights, see figure 2.10 and figure 2.11.

The grey line represents the M1 model prediction result of orders from September 2016 to February 2017, which is the naive benchmark. The orange line is the M2 model prediction result, which is generated by ARIMA model and given by some experts in the field. The green line shows the M3 model prediction result, which exerts some deep learning models. The blue line shows the actual values. And the red vertical line separates training set and test set. As we can see, In figure 2.10,

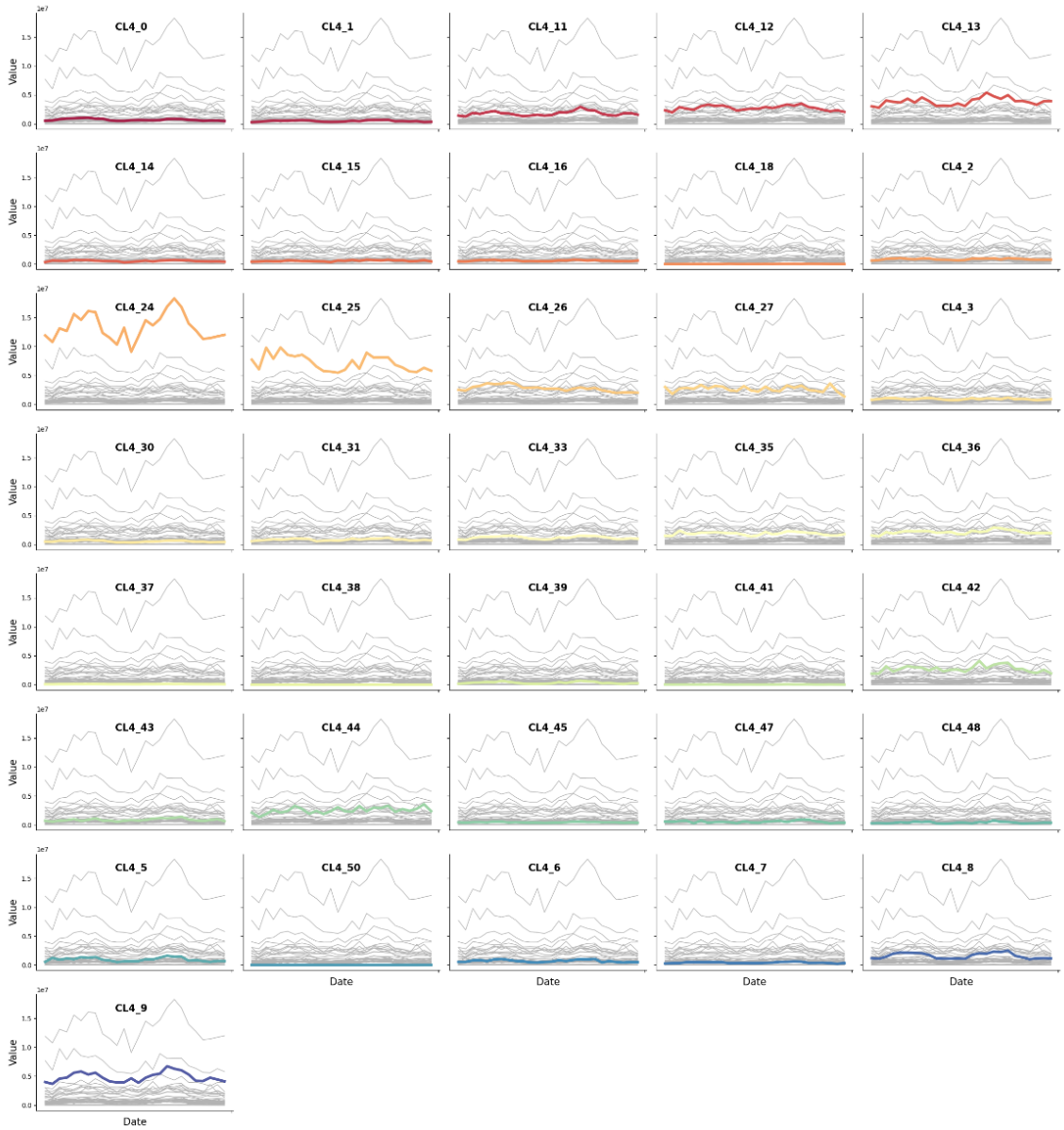


Figure 2.8: Clustering C Graph

	CL4	ACC_m3	ACC_m2	Acc_m3 - Acc_m2
0	CL4_17	0.000000	1.000000	-1.000000
1	CL4_19	0.000000	0.000000	0.000000
2	CL4_20	0.790065	0.805607	-0.015542
3	CL4_22	0.000000	1.000000	-1.000000
4	CL4_28	0.052141	0.000000	0.052141
5	CL4_32	0.000000	0.000000	0.000000
6	CL4_34	0.917770	0.890536	0.027234
7	CL4_4	0.810283	0.857311	-0.047027
8	CL4_40	0.869539	0.928122	-0.058583
9	CL4_46	0.819505	0.875255	-0.055750
10	CL4_49	0.889639	0.868410	0.021229

Figure 2.9: M3 accuracy Vs. M2 accuracy

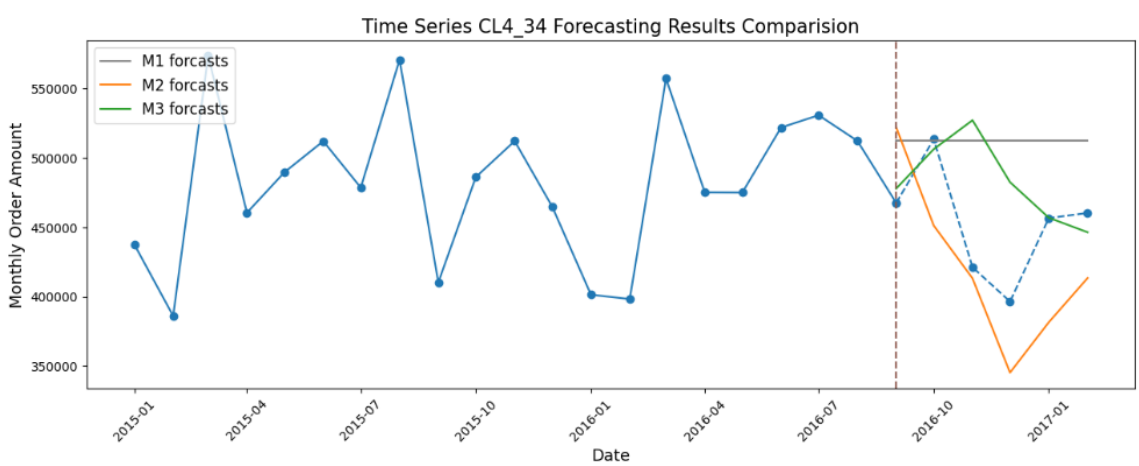


Figure 2.10: Time Series CL4_34 Prediction Results

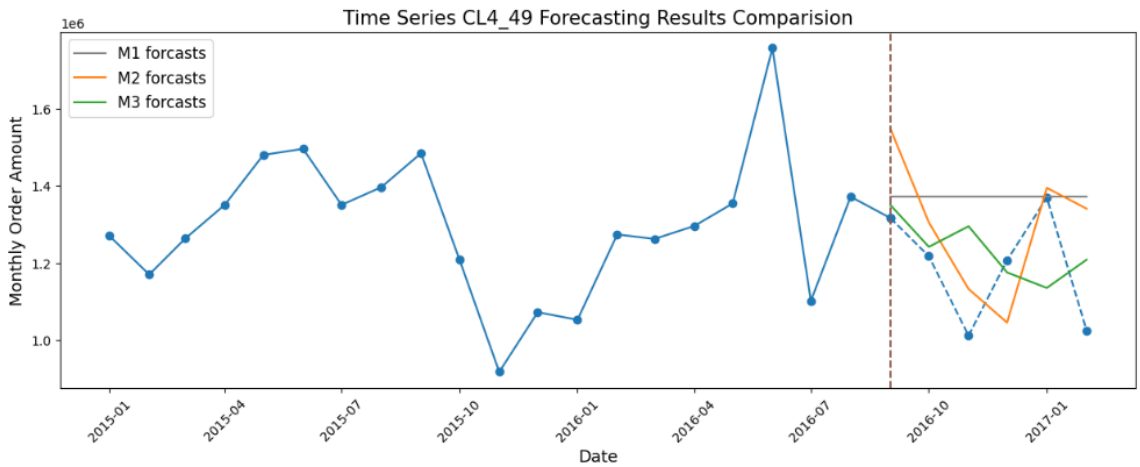


Figure 2.11: Time Series CL4_49 Prediction Results

although both M2 and M3 captures part of the time series pattern, M2 accuracy is lower than M3 due to its wrong scale. The M3 is closer to the true values and M2 underestimates true values. this shows that deep learning has the exceptional advantage of capturing complex relationships. M1 is missing all the information contained in the training data except the order value of August 2016. It's like a random guess. We will mainly focus on observing M2 and M3 in figure 2.11. In this figure, although M2 captures the trend of time series CL4_49 better than M3, M2 accuracy is lower than M3. This is because the predictions of M3 are closer to the true values. Again, it proves that deep learning is good at extracting more features in time series data. Hence, we can see the promise of deep learning in improving time series forecasting and we will use time series in next chapter for hierarchical time series forecasting.

2.4 Summary

In this chapter, we first introduce the existing methods of clustering and classification. Then we propose our distance measurement approach, added-value method. Then we construct a novel framework in which decisions such as the number of clusters and prediction based on added-values are made using different techniques. Later, we use real-world data to perform numerical studies with three models to demonstrate the added-value framework in time series classification and clustering. The added-value approach is easy to implement and it helps us to target 11 time series that has a large potential to increase the prediction accuracy with little effort and time. Although we set the threshold

and number of clusters manually here, in the future, we can generate an automated decision-making pipeline to train a model to cluster based on the nature of data.

Chapter 3

Hierarchical Time Series Forecasting

3.1 Introduction

In this chapter, we dive into hierarchical time series analysis. When we encounter a collection of time series in which one can be aggregated from or disaggregated from others, it could be either a hierarchical or grouped time series. Hierarchical time series has a clear hierarchical structure, such as geographical splitting, which means the lower levels are nested within the higher-level groups. We only have one way to aggregate the data correctly, which should follow the hierarchical structure. Grouped time series has crossed instead of nested levels. There can be several ways to aggregate the data [5].

We should be aware of the difference between these two types of time series because some methodologies can be only applied to hierarchical time series, such as the top-down method. We are focusing on hierarchical time series (HTS) forecasting here.

HTS is a collection of time series with a nested structure. A time series in the hierarchical structures could not be an isolated one. It is either a mother of some time series in the structure, a child of some time series in the structure, or both. Predicting hierarchical time series has always been challenging due to the inherently complex relationship between all the time series. Let's use the graph below to start exploring HTS forecasting.

Figure 3.1 is a 2-level hierarchical tree. Level 0 is a company's daily total sales, which is the most aggregated level. Level 1 is the sales in two cities where the company owns a business. The sum of city A and city B in level 1 equals the value of the Total in level 0. Level 2 is the sales in different stores in the two cities. Similarly, the sum of store A1, store A2, and store A3 in level 2 equals the value of city A in level 1. The sum of store B1 and store B2 in level 2 equals the value of

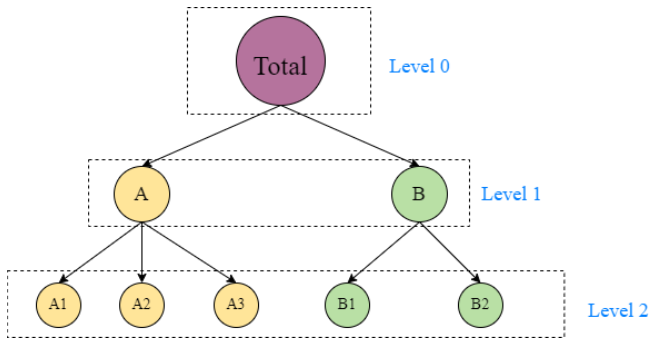


Figure 3.1: A 2-level Hierarchical Tree Diagram Example

city B in level 1. For the history data, it makes sense that

$$A1 + A2 + A3 = A, B1 + B2 = B, A + B = Total.$$

However, when we do time series forecasting, it is hard to maintain the forecasts consistency that exists in the historical data. It is very likely that

$$\widehat{A1} + \widehat{A2} + \widehat{A3} \neq \widehat{A}, \widehat{B1} + \widehat{B2} \neq \widehat{B}, \widehat{A} + \widehat{B} \neq \widehat{Total},$$

where we use hat symbol to denote forecasts that are generated independently at various levels.

Let's make it more general. Suppose we have an n-level hierarchy, level 0 is the top level and the most-aggregated level, and level n is the bottom and most-disaggregated level. The hierarchy relationship between HTS can be explained with the following math function:

$$Y(t) = S \cdot Y_B(t)$$

Let's illustrate the HTS function with Figure 3.1. $Y(t)$ is an n-dimensional ($n = 8$ in this case) vector of time series values in all levels at time t. There are eight time series in it :

$$y(Total, t), y(A, t), y(B, t), y(A1, t), y(A2, t), y(A3, t), y(B1, t), \text{and } y(B2, t),$$

which contains a tree structure among them. $y(Total, t)$ is the value at time t of time series at level 0, which is the most-aggregated level. $y(A, t)$ and $y(B, t)$ are the values at time t of time series at level 1, which is the second-most (or middle) aggregated level. The time series values at level 0 at time t are the summation of the time series values at level 1 at time t, and

$$y(Total, t) = y(A, t) + y(B, t)$$

Table 3.1: Term Description of HTS Function

Term	Description
$Y(t)$	n-dimensional vector of all-level time series historical value at time t
$y(X,t)$	time series historical value of node X at time t
S	summing matrix that contains the tree relationship
$Y_B(t)$	m-dimensional vector of bottom-level time series historical value at time t
X	all nodes
m	the number of time series in the bottom level
n	the number of time series of all levels

$y(A1,t), y(A2,t), y(A3,t), y(B1,t),$ and $y(B2,t)$ is the time-t value of time series at level 2, which is the most-disaggregated level. We can separate the time series in level 2 into two kinds based on the nodes in level 1: node A-related time series and node B-related time series. The relationship between the time series in level 1 and level 2 is as follows :

$$y(A,t) = y(A1,t) + y(A2,t) + y(A3,t)$$

and

$$y(B,t) = y(B1,t) + y(B2,t)$$

In addition, we can trace back to level 0 from level 2.

$$y(Total,t) = y(A,t) + y(B,t) = y(A1,t) + y(A2,t) + y(A3,t) + y(B1,t) + y(B2,t)$$

S is the summing matrix of size $n \times m$ (8×5 in this case) which shows the binary representation of all-level time series. The table below shows how to generate the summing matrix S.

For example, Since $y(Total,t) = 1 \times y(A1,t) + 1 \times y(A2,t) + 1 \times y(A3,t) + 1 \times y(B1,t) + 1 \times y(B2,t)$, the row of $y(Total,t)$ is (1 1 1 1 1).

	$y(A1,t)$	$y(A2,t)$	$y(A3,t)$	$y(B1,t)$	$y(B2,t)$
$y(Total,t)$	1	1	1	1	1
$y(A,t)$	1	1	1	0	0
$y(B,t)$	0	0	0	1	1
$y(A1,t)$	1	0	0	0	0
$y(A2,t)$	0	1	0	0	0
$y(A3,t)$	0	0	1	0	0
$y(B1,t)$	0	0	0	1	0
$y(B2,t)$	0	0	0	0	1

The Figure 3.1 can be represented as the following equations.

$$\underbrace{\begin{pmatrix} y(total,t) \\ y(A,t) \\ y(B,t) \\ y(A1,t) \\ y(A2,t) \\ y(A3,t) \\ y(B1,t) \\ y(B2,t) \end{pmatrix}}_{Y(t)} = \underbrace{\begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}}_{S} \cdot \underbrace{\begin{pmatrix} y(A1,t) \\ y(A2,t) \\ y(A3,t) \\ y(B1,t) \\ y(B2,t) \end{pmatrix}}_{Y_B(t)}$$

Moreover,

$$\underbrace{\begin{pmatrix} y(total,t) \\ y(A,t) \\ y(B,t) \\ y(A1,t) \\ y(A2,t) \\ y(A3,t) \\ y(B1,t) \\ y(B2,t) \end{pmatrix}}_{Y(t)} = \underbrace{\begin{pmatrix} y(A1,t)+y(A2,t)+y(A3,t)+y(B1,t)+y(B2,t) \\ y(A1,t)+y(A2,t)+y(A3,t) \\ y(B1,t)+y(B2,t) \\ y(A1,t) \\ y(A2,t) \\ y(A3,t) \\ y(B1,t) \\ y(B2,t) \end{pmatrix}}_{S \times Y_B(t)}$$

However, the hierarchical relationship between levels can't be guaranteed when it comes to prediction. For instance, the sum of the prediction value of time series A (level 1) and time series

B (level 1) at time t does not equal time series $Total$ (level 0) at time t. The forecasts could be more consistent across different levels. However, aggregate consistency is one of the usual requirements in making decisions for good hierarchical time series forecasting. In other words, the forecast should be coherent. Luckily, there are several ways to maintain consistency. They aim to generate coherent forecasts for all levels of aggregation.

3.2 Four Common Approaches

3.2.1 Bottom-up Approach

We first forecast the bottom level of the hierarchy and then add the corresponding time series results for higher levels based on the tree structures. The advantage of this approach is that it maintains all the information at the bottom level. The disadvantages are missing relationships between the series, poor higher-level performance, heavy computation tasks, and high sensitivity to noise [5].

Let's still use the Figure 3.1 as an example. We first predict each store's sales (bottom-level time series) and get the prediction values: $\widehat{A1}, \widehat{A2}, \widehat{A3}, \widehat{B1}, and \widehat{B2}$. Then according to the tree structure, we add the prediction values of A1, A2, and A3 to generate the expected sales of city A. Similarly, we add the prediction values of B1 and B2 to generate the expected sales of city B. Then add the aggregated expected sales of city A and the aggregated expected sales of city B together to get the prediction of total sales for this company.

We want to achieve

$$\widetilde{Y}(t) = S \cdot P(BU) \cdot \widehat{Y_B}(t),$$

where we use tilde symbol to denote revised (or reconciled) forecasts at various levels.

We first forecast time series in the lowest level at time t in the hierarchy and then add the prediction results based on the hierarchy structure to get the prediction values for higher levels. Take the Figure 3.1 as example, The lowest level in this 2-level hierarchical tree structure is level 2 and each time series represents the sales of individual stores. Therefore, we apply an appropriate methodology to predict each store's sales at time t and get the prediction values: $\widehat{y}(A1, t)$, $\widehat{y}(A2, t)$, $\widehat{y}(A3, t)$, $\widehat{y}(B1, t)$, $\widehat{y}(B2, t)$. Then according to the tree structure, we add the prediction values of A1, A2, and A3 to generate the predicted sales of city A. $\widetilde{y}(A, t)$ = $\widehat{y}(A1, t)$ + $\widehat{y}(A2, t)$ + $\widehat{y}(A3, t)$

Similarly, we add the prediction values of B1 and B2 to generate the predicted sales of city B. $\widetilde{y}(B,t)$ = $\widehat{y}(B1,t)$ + $\widehat{y}(B2,t)$

Then add the predicted sales of city A and the predicted sales of city B together to get the prediction of total sales for this company. $\widetilde{y}(Total,t)$ = $\widetilde{y}(A,t)$ + $\widetilde{y}(B,t)$

Table 3.2: Term Description I

Term	Description
$\widetilde{Y}(t)$	n-dimensional vector of all-level time series reconciled prediction values at time t
$\widetilde{y}(X,t)$	time series reconciled prediction values of node X at time t
$\widehat{Y}(t)$	n-dimensional vector of all-level time series independent prediction values at time t
$\widehat{y}(X,t)$	time series independent prediction values of node X at time t
$P(BU)$	binary matrix that represents only bottom-level time series
$\widehat{Y_B}(t)$	m-dimensional vector of bottom-level time series independent prediction values at time t
S, X, m, n	same as table 3.1

Below is the equation representation of bottom-level approach for Figure 3.1 :

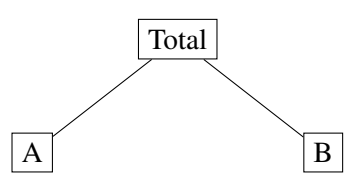
$$
\begin{aligned}
\widetilde{Y}(t) &= S \cdot P(BU) \cdot \widehat{Y_B}(t) \\
&= \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix} \times \begin{pmatrix} 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix} \times \begin{pmatrix} \widehat{y}(A1,t) \\ \widehat{y}(A2,t) \\ \widehat{y}(A3,t) \\ \widehat{y}(B1,t) \\ \widehat{y}(B2,t) \end{pmatrix} \\
&= \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix} \times \begin{pmatrix} \widehat{y}(A1,t) \\ \widehat{y}(A2,t) \\ \widehat{y}(A3,t) \\ \widehat{y}(B1,t) \\ \widehat{y}(B2,t) \end{pmatrix} \\
&= \begin{pmatrix} \widehat{y}(A1,t) + \widehat{y}(A2,t) + \widehat{y}(A3,t) + \widehat{y}(B1,t) + \widehat{y}(B2,t) \\ \widehat{y}(A1,t) + \widehat{y}(A2,t) + \widehat{y}(A3,t) \\ \widehat{y}(B1,t) + \widehat{y}(B2,t) \\ \widehat{y}(A1,t) \\ \widehat{y}(A2,t) \\ \widehat{y}(A3,t) \\ \widehat{y}(B1,t) \\ \widehat{y}(B2,t) \end{pmatrix}
\end{aligned}
\tag{3.1}
$$

So,

$$\widetilde{Y}(t) = \begin{pmatrix} \widetilde{y}(total,t) \\ \widetilde{y}(A,t) \\ \widetilde{y}(B,t) \\ \widetilde{y}(A1,t) \\ \widetilde{y}(A2,t) \\ \widetilde{y}(A3,t) \\ \widetilde{y}(B1,t) \\ \widetilde{y}(B2,t) \end{pmatrix} = \begin{pmatrix} \widehat{y}(A1,t) + \widehat{y}(A2,t) + \widehat{y}(A3,t) + \widehat{y}(B1,t) + \widehat{y}(B2,t) \\ \widehat{y}(A1,t) + \widehat{y}(A2,t) + \widehat{y}(A3,t) \\ \widehat{y}(B1,t) + \widehat{y}(B2,t) \\ \widehat{y}(A1,t) \\ \widehat{y}(A2,t) \\ \widehat{y}(A3,t) \\ \widehat{y}(B1,t) \\ \widehat{y}(B2,t) \end{pmatrix}$$

3.2.2 Top-down Approach

In the top-down approach, we forecast the top level of the hierarchy and then split the results for lower levels by the historic proportions, which is the most common way. The advantages are that it is the most straightforward approach, performs well on higher levels, and only needs a single forecast. The disadvantages are poor lower-level performance due to information loss caused by historical proportions split[5]. Let us take a simple hierarchy time series, which consists of level 0 and level 1 in 3.1. We predict the total sales first. Then, based on each store's sales contribution, get the prediction values: A and B. The new figure is as below:



The mathematical representation of the top-down method is as following:

$$
\begin{aligned}
\widetilde{Y}(t) &= S \times P(TD) \times \widehat{Y_T}(t) \\
&= \begin{pmatrix} 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix} \times \begin{pmatrix} p(A) & 0 & 0 \\ p(B) & 0 & 0 \end{pmatrix} \times \begin{pmatrix} \widehat{y}(total,t) \\ \widehat{y}(total,t) \\ \widehat{y}(total,t) \end{pmatrix} \\
&= \begin{pmatrix} p(A)+p(B) & 0 & 0 \\ p(A) & 0 & 0 \\ p(B) & 0 & 0 \end{pmatrix} \times \begin{pmatrix} \widehat{y}(total,t) \\ \widehat{y}(total,t) \\ \widehat{y}(total,t) \end{pmatrix} \\
&= \begin{pmatrix} p(A) \cdot \widehat{y}(total,t) + p(B) \cdot \widehat{y}(total,t) \\ p(A) \cdot \widehat{y}(total,t) \\ p(B) \cdot \widehat{y}(total,t) \end{pmatrix}
\end{aligned} \tag{3.2}
$$

And,

$$p(A) + p(B) = 1$$

Table 3.3: Term Description II

Term	Description
$\widetilde{Y}(t), \widehat{y}(X,t)$	same as table 3.2
$P(TD)$	binary matrix that represents only the most-aggregated level time series
$\widehat{Y_T}(t)$	(n-m)-dimensional vector of most-aggregated level time series independent prediction values at time t
S, X, m, n	same as table 3.1

The contribution of A and B (denoted by $p(A)$ and $p(B)$) can be calculated from historical values or forecast values [6]. Let's take historical values as an example. There are two general ways to calculate the P matrix.

3.2.2.1 Average historical proportions

The first way is called average historical proportions. Each p_i is the ratio of the average value of one time series across all time to average value of the most aggregate time series value across all time.

$$p_i = \Big(\sum_{t=1}^{n} \frac{y(i,t)}{n}\Big) / \Big(\sum_{t=1}^{n} \frac{y(total,t)}{n}\Big)$$

For example,

	t=1	t=2	t=3	t=4
y(Total, t)	4	8	16	4
y(A, t)	1	4	2	1
y(B,t)	3	4	14	3

The average value of the time series A across all time is

$$\begin{aligned} \sum_{t=1}^{n} \frac{y(A,t)}{n} &= \Big(\frac{y(A,1)+y(A,2)+y(A,3)+y(A,4)}{4}\Big) \\ &= \frac{1+4+2+1}{4} \\ &= 2 \end{aligned} \tag{3.3}$$

The average value of the time series B across all time is

$$\begin{aligned} \sum_{t=1}^{n} \frac{y(B,t)}{n} &= \Big(\frac{y(B,1)+y(B,2)+y(B,3)+y(B,4)}{4}\Big) \\ &= \frac{3+4+14+3}{4} \\ &= 6 \end{aligned} \tag{3.4}$$

The average value of the time series $Total$ across all time is

$$\begin{aligned} \sum_{t=1}^{n} \frac{y(Total,t)}{n} &= \Big(\frac{y(Total,1)+y(Total,2)+y(Total,3)+y(Total,4)}{4}\Big) \\ &= \frac{4+8+16+4}{4} \\ &= 8 \end{aligned} \tag{3.5}$$

the result is

$$\begin{aligned} p(A) &= \Big(\sum_{t=1}^{n} \frac{y(A,t)}{n}\Big) / \Big(\sum_{t=1}^{n} \frac{y(total,t)}{n}\Big) \\ &= \frac{2}{8} \\ &= 0.25 \end{aligned} \tag{3.6}$$

and

$$\begin{aligned} p(B) &= \Big(\sum_{t=1}^{n} \frac{y(B,t)}{n}\Big) / \Big(\sum_{t=1}^{n} \frac{y(total,t)}{n}\Big) \\ &= \frac{6}{8} \\ &= 0.75 \end{aligned} \tag{3.7}$$

We can also get p(A) + p(B) = 1

3.2.2.2 Proportions of historical averages

The second method is the proportions of the historical averages.

$$p(i) = \frac{1}{n}\Big(\sum_{t=1}^{n} \frac{y(i,t)}{y(Total,t)}\Big)$$

$$\begin{aligned} p(A) &= \frac{1}{4}\Big(\frac{y(A,1)}{y(Total,1)} + \frac{y(A,2)}{y(Total,2)} + \frac{y(A,3)}{y(Total,3)} + \frac{y(A,4)}{y(Total,4)}\Big) \\ &= \frac{1}{4}\Big(\frac{1}{4} + \frac{4}{8} + \frac{2}{16} + \frac{1}{4}\Big) \\ &= \frac{1}{4} \cdot \frac{9}{8} \\ &= 0.28 \end{aligned} \tag{3.8}$$

Each p_i is the average ratio of one lower-level time series to the most-aggregated time series at all different time t.

3.2.3 Middle-out Approach

We first forecast the middle level of the hierarchy. Then forecast the lower levels by the top-down approach and the higher levels by the button-up approach. It doesn't lose much information and has fewer computation tasks. However, we need to decide where the middle level is.

3.2.4 Optimal Reconciliation Approach

We first forecast each of the series at all levels of the hierarchy. Then use a linear regression model to reconcile all the forecasts. The advantages it is that the performance is accurate and unbiased, the relationship between time series remains, and each level can apply different forecasting methods. However, the computation of this method is heavy [5].

3.3 Framework of Reconciliation with Neural Network

As mentioned above, these four popular approaches have disadvantages in either the methodology perspective or the implementation step. For instance, the top-down approach ignores the latent information of individual time series when forecasting values for less-aggregate-level time series. The optimal reconciliation approach must be simplified to apply to time series at scale. Moreover, these four approaches are all based on pre-generated forecasts at all levels, which means that the prediction for individual time series (base prediction) is first made and then further reconciled across all levels with the base predictions. The reconciliation in standard hierarchical time series forecast methods is a linear approach, which is too simple to describe hierarchical structures. Therefore, machine learning techniques such as deep neural networks have been applied to generate more accurate time series data. The advantages of deep neural networks are that they can construct a non-linear model, and it doesn't require data to meet the regression assumptions, such as identical independent distribution.

Some hierarchy time series forecasting-related papers successfully adopt the neural networks as part of the procedure. Some authors introduce a top-down approach with deep neural networks and add the differences between the actual values and predictions of lower-level time series as a penalty term to the loss function to ensure the projections are coherent[7]. Some other authors apply a neural network to generate the bottom-level time series forecasting with the base forecasts of all-level time series. Then use a standard bottom-up approach to get the reconciled all-level time series forecasts[8].

However, the computation cost can be high since the paper[7] only takes two consecutive level time series each time and has a complex loss function. Article[8] proposed approach is based on pre-generated forecasts that don't grasp the relationship between time series.

To generate a cost-efficient coherent forecast for hierarchy time series data, we propose a new method called reconciliation with neural network (R-NN). The graph below is the entire

procedure of our method, R-NN.

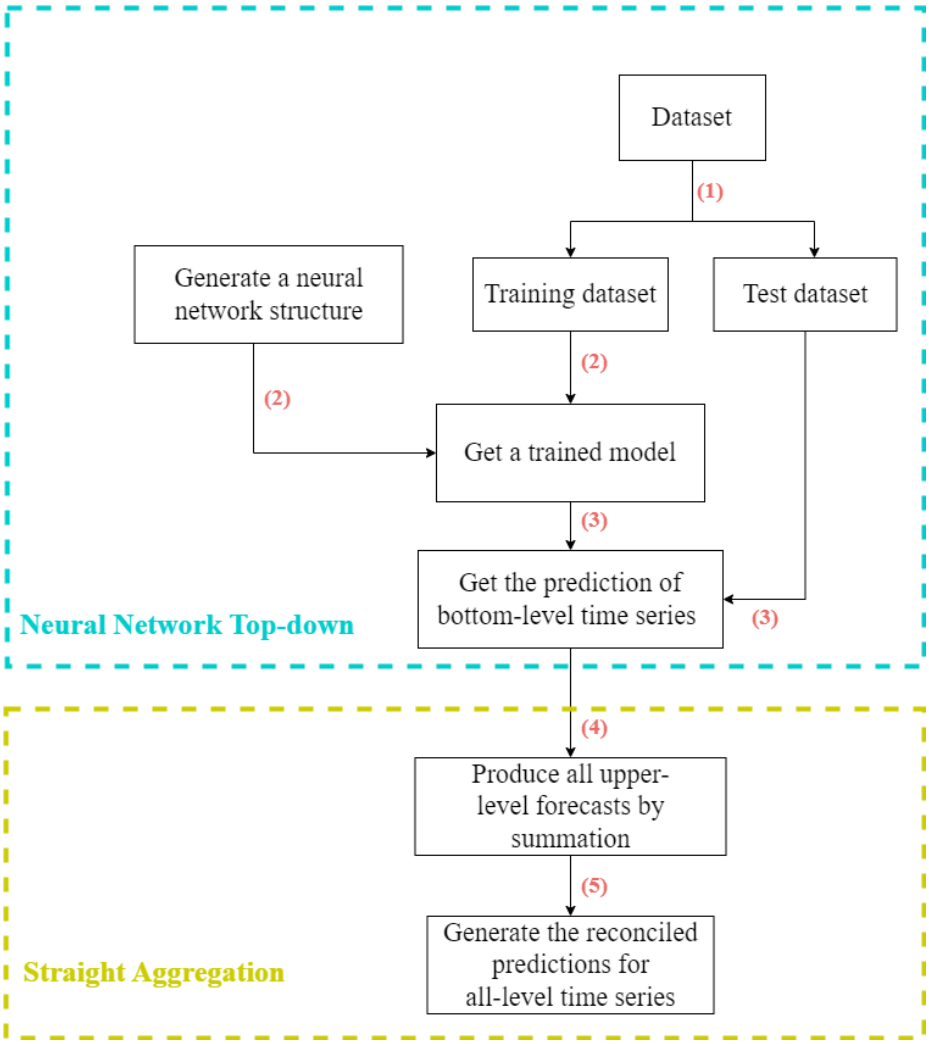


Figure 3.2: Reconciliation With Neural Network (R-NN) Framework

There are two parts to the whole process, the first part is to use a neural network to generate the bottom-level time series prediction with all the aggregated-level time series, and the second part is to get the reconciled predictions of all-level time series by straightly summation. R-NN method has many advantages compared with popular hierarchy time series forecasting approaches. First, time series are predicted individually in a method based on pre-generated forecasts, they don't consider the relationship between time series, resulting in biased forecasts. So, instead of using base predictions, R-NN utilizes deep neural networks on raw data, which contains first-hand information, to capture the non-linear relationship of each bottom-level time series and other aggregate-level time

series. Second, since the bottom-level time series is the integrated representation of all upper-level time series, we can aggregate straightly to generate predictions for upper-level time series, which is cost-efficient and accurate. Now, let's explain the five-step R-NN algorithm with a m-level HTS, see Figure 3.3.

Assume there are m levels, each level has n_m time series, each time series has t periods, and the t periods consist of $t1$ training periods and $t2$ testing periods. For instance, the first level HTS has $n1$ time series, and the second level HTS has $n2$ time series, $\text{TS}_{\text{L}m,i}(t1)$ is the training values of i-th time series in m-th level over one HTS

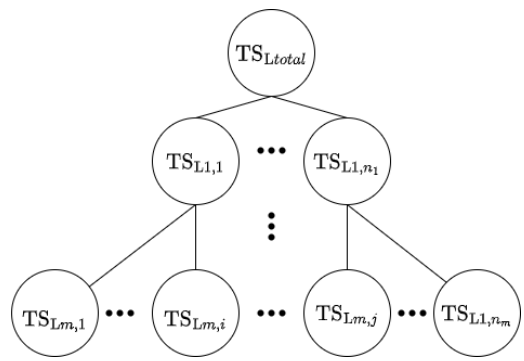


Figure 3.3: M-level Hierarchical Time Series (HTS)

Neural Network Top-down approach forms a non-linear regression model. The dependent variable is individual bottom-level time series and independent variables are all-level time series except the bottom-level time series.

$$\text{TS}_{\text{L}m,i} = f(\text{TS}_{\text{L}total}, \text{TS}_{\text{L}1,1}, \text{TS}_{\text{L}1,2}, ..., \text{TS}_{\text{L}m-1,n_{m-1}}) + \epsilon$$

1. Split the each time seires,$\text{TS}_{\text{L}total}(t), \text{TS}_{\text{L}1,1}(t)...\text{TS}_{\text{L}m,n_m}(t)$ into training set:

   $$\text{TS}_{\text{L}total}(t1), \text{TS}_{\text{L}1,1}(t1), ..., \text{TS}_{\text{L}m,n_m-1}(t1), \text{TS}_{\text{L}m,n_m}(t1),$$

   and test set:

   $$\text{TS}_{\text{L}total}(t2), \text{TS}_{\text{L}1,1}(t2), \text{TS}_{\text{L}1,2}(t2), ..., \text{TS}_{\text{L}m,n_m-1}(t2), \text{TS}_{\text{L}m,n_m}(t2)$$

   .

2. First, split the training dataset into input data and output data. Input is the all aggregated-level time series (all-level time series expect bottom-level time series):

$$\mathrm{TS}_{\mathrm{L}total}(t1), \mathrm{TS}_{\mathrm{L}1,1}(t1), \mathrm{TS}_{\mathrm{L}1,2}(t1), ..., \mathrm{TS}_{\mathrm{L}m-1,n_{m-1}}(t1),$$

and output is the bottom-level time series:

$$\mathrm{TS}_{\mathrm{L}m,1}(t1), \mathrm{TS}_{\mathrm{L}m,2}(t1), ..., \mathrm{TS}_{\mathrm{L}m,n_m}(t1).$$

Then use the input and output to train a generated neural network,

$$\mathrm{TS}_{\mathrm{L}m,i}(t1) = f(\mathrm{TS}_{\mathrm{L}total}(t1), \mathrm{TS}_{\mathrm{L}1,1}(t1), \mathrm{TS}_{\mathrm{L}1,2}(t1), ..., \mathrm{TS}_{\mathrm{L}m-1,n_{m-1}}(t1)) + \epsilon,$$

and save the parameters for later forecasting step.

3. First, similar to splitting the training dataset, we split the testing dataset into input and output. Then recall the trained model in step 2, and use it to predict the bottom-level time series values of desired windows,

$$\widehat{\mathrm{TS}}_{\mathrm{L}m,i}(t2) = f(\mathrm{TS}_{\mathrm{L}total}(t2), \mathrm{TS}_{\mathrm{L}1,1}(t2), \mathrm{TS}_{\mathrm{L}1,2}(t2), ..., \mathrm{TS}_{\mathrm{L}m-1,n_{m-1}}(t2)) + \epsilon.$$

4. Add up the bottom-level time series forecasts straightly based on the hierarchy structure. For instance,

$$\widetilde{\mathrm{TS}}_{\mathrm{L}total}(t) = \widehat{\mathrm{TS}}_{\mathrm{L}m,1}(t) + \widehat{\mathrm{TS}}_{\mathrm{L}m,2}(t) + ... + \widehat{\mathrm{TS}}_{\mathrm{L}m,n_m}(t)$$

5. Assign the reconciled all-level time series as the forecast result.

3.4 Numerical Analysis

3.4.1 Goal of This Study

The goal of this study is to improve the overall or systematic accuracy of time series forecasting. We show that reconciliation with neural network (R-NN) is an outstanding approach compared with other methods.

3.4.2 Dataset

The dataset we use in this study consists of the monthly sales of a retail company. This dataset consists of 60 time series, and they construct a two-level HTS. The bottom level (level 2) has 51-time series, each representing the monthly sales from January 2015 to February 2017 in 51 states for some retail companies. And they aggregate based on the regions of a country that the state locates. Moreover, the dataset includes various products and customer levels, such as customer channels and regions. We further process the raw data and obtain target data with the following hierarchical levels: nation (level 0), channels (level 1), and regions (level 2). For instance, supermarket sales in the US northeastern region can be aggregated accordingly. After aggregation, there are eight time series in level 1, and they contain the monthly sales information from January 2015 to February 2017 in 8 channels for some retail companies. Then the most-aggregate level has one time series, and each value is the sum of the time series in level 1 of each month.

Level	**Number of Time Series**	**Time Series Component**
Total US (0)	1	1
Channel (1)	8	8
Region (2)	51	2+9+18+15+2+1+2+2

Table 3.4: HTS Dataset Summary

3.4.3 Data Preprocessing

We impute the missing values with 0 since the missing values in this dataset mean no sale. In addition, since the input of neural networks is time series at different levels, the scale of the values is very different. To reduce the effect of the scale of values while training models, we normalize all the time series values before fitting them into a neural network. We find the maximum and minimum deals of all the time series. Then, we subtract the minimum value from each value in all the time series and divide the result by the difference between the maximum and minimum. Moreover, when we predict the sales in basic units, we perform the inverse of normalization on it.

3.4.4 Model

We use two models in numerical analysis. The first model is a naive forecast that uses the actual value in the previous period as the current forecast. This model provides a baseline with which

we compare the neural network model. The second model is the proposed R-NN approach. There are five steps to implement the approach.

3.4.4.1 Bottom-level time series prediction with R-NN model

First, we split each time series into training and test sets. Each time series has 26 time-sorted values, the monthly sales from January 2015 to February 2017. We pick the first 24 values, the monthly sales from 2015 January to 2016 December, as the training set, and the last two values, the monthly sales from 2017 January to 2017 February, as the test set.

Figure 3.4: Time Series Training and Testing Sets

Second, we split the training set into input and output sets. Input is the all aggregated-level

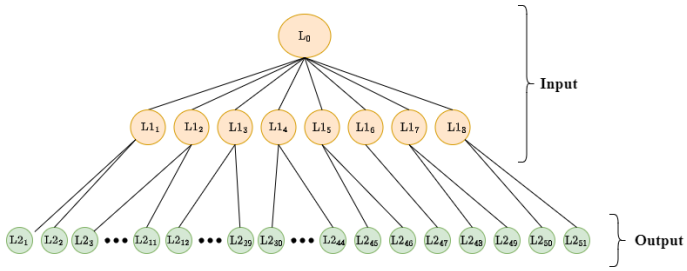


Figure 3.5: R-NN Model Input and Ouput

time series (nation-level and channel-level) of the training set, and output is the bottom-level time series (state-level) of the training set, see Figure 3.5.

Then use the input and output to train a neural network and save the parameters for later forecasting step. To simplify the process, The neural network we picked here is a simple fully-

connected network that has one input layer with 9 nodes, one hidden layer with 1000 nodes, and one output layer with 51 nodes, see Figure 3.6. The number of nodes for the input layer is the same as the number of time series in all-aggregated levels (1 time series in level 0 and 8 time series in level 1 for our dataset). The number of nodes for the output layer is the same as the number of time series in the bottom level (51 time series in level 2 for our dataset). In addition, the activate function is Relu and the loss function is Mean squared error. We minimize the loss function with Adam optimizer, an efficient mini-batch stochastic optimization that only requires first-order gradients.

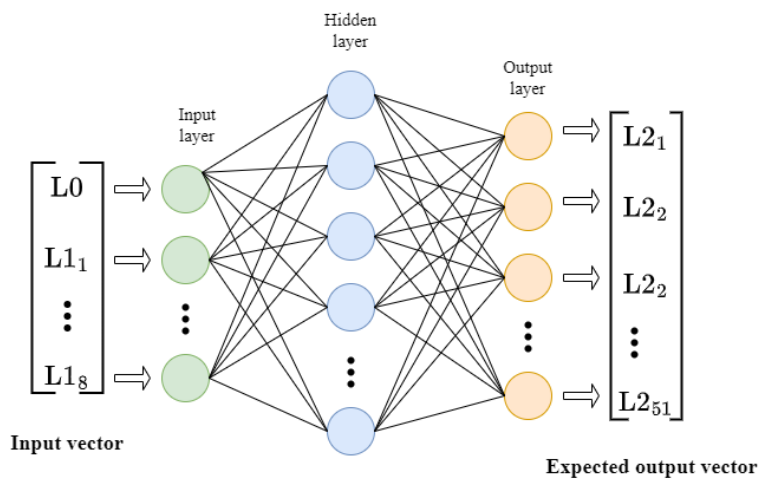


Figure 3.6: R-NN Model Training Process

Third, we split the testing set into input and output sets. Input is the all aggregated-level time series of the test set. Then recall the trained model in the previous step, and use the input to predict the bottom-level(region-level) time series values of January 2017 and February 2017, see Figure 3.7.

3.4.4.2 Straight aggregation

Fourth, We add up the region-level time series forecasts to generate the channel-level and nation-level forecasts based on the hierarchy structure. For instance, the first region time series at channel-level consists of two time series at region-level. The predicted monthly sale of 2017 January in channel 1 is the sum of the predicted monthly sales of 2017 January in region 1 and 2.

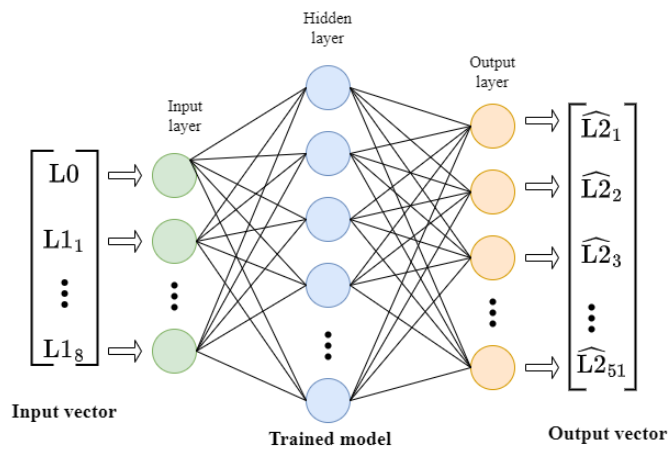


Figure 3.7: R-NN Model Prediction Process

$$\begin{bmatrix} \widetilde{\mathrm{L0}} \\ \widetilde{\mathrm{L1}}_1 \\ \widetilde{\mathrm{L1}}_2 \\ \vdots \\ \widetilde{\mathrm{L2}}_{51} \end{bmatrix} = \begin{bmatrix} 1 & 1 & \cdots & 1 \\ 1 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 \end{bmatrix} \times \begin{bmatrix} \widehat{\mathrm{L2}}_1 \\ \widehat{\mathrm{L2}}_2 \\ \vdots \\ \widehat{\mathrm{L2}}_{51} \end{bmatrix} \tag{3.9}$$

Fifth, after the summation calculations based on the hierarchy structure, we consider the results as reconciled forecasts of time series at all levels accordingly.

$$\begin{bmatrix} \widehat{\mathrm{L0}} \\ \widehat{\mathrm{L1}}_1 \\ \widehat{\mathrm{L1}}_2 \\ \vdots \\ \widehat{\mathrm{L2}}_{51} \end{bmatrix} = \begin{bmatrix} \widetilde{\mathrm{L0}} \\ \widetilde{\mathrm{L1}}_1 \\ \widetilde{\mathrm{L1}}_2 \\ \vdots \\ \widetilde{\mathrm{L2}}_{51} \end{bmatrix} \tag{3.10}$$

3.4.5 Evaluation Method

The evaluation method is the same as the previous chapter.

3.4.6 Result

We compare the systematic forecasting performance of our proposed method, R-NN, with two baseline models. In the baseline 1 method, we use one-step ahead naive model to generate time

series prediction at all levels. For example, the actual orders of CL4_29 in 2016 December is the naive forecast for this series in 2017 January. Moreover, the actual orders of CL2_8 in 2016 December is the naive forecast for this series in 2017 January. In the baseline 2 method, we use ARIMA model to generate bottom-level time series predictions and then aggregate straightly to get all upper-level predictions. Table 3.5 summarizes the accuracy of different methods at the bottom-level and all-level.

Model	**Systematic TS Accuracy**	**Bottom-level TS Accuracy**
R-NN	96%	91%
Baseline 1	91%	88%
Baseline 2	95%	92%

Table 3.5: Systematic Prediction Summary

We can see from the result that our proposed model performance wins in both categories, see table 3.5. Compared with the baseline 1 and 2 model prediction accuracy, the R-NN method improves the overall accuracy across all hierarchical levels by 5% and 1% respectively. In addition, although both baseline 2 method and R-NN get the predictions of bottom-level first and apply straight aggregation, R-NN model has the biggest increase when comparing the systematic and the bottom-level time series forecasts. This is because our proposed R-NN uses all the aggregated-level time series information when generating the bottom-level time series forecasts. Therefore, when we aggregate straightly for reconciliation, we have considered the relationship between series.

Compared with the baseline 1 and 2 models, another reason that lends to the improvement of the systematic accuracy is we use the raw dataset to train the neural network instead of individual forecasts of each time series. Neural networks are good at extracting the relationship between input and output variables since it can construct non-linear relationship between them. We take advantage of this good feature to learn the complex relationship between bottom-level time series and all-upper-level time series. Therefore, we can generate accurate and unbiased predictions.

3.5 Summary

In this chapter, we forecast time series at scale with hierarchical structures. We first review four standard hierarchical time series forecasting methods: bottom-up, top-down, middle-out, and optimal reconciliation. Then we propose a new approach-reconciliation with neural network (R-NN) for hierarchical forecasting which takes non-linear relationship between time series into consideration

for forecasting. Later, we perform numerical studies to show the efficiency of our approach. The systematic accuracy of overall 60 time series improved by 5% and 1% compared with the accuracy that is based on naive method and bottom-up approach.

Bibliography

[1] A. Javed, B. S. Lee, and D. M. Rizzo, "A benchmark study on time series clustering," *Machine Learning with Applications*, vol. 1, p. 100001, 2020.

[2] A. Amidon, "A brief survey of time series classification algorithms," Aug 2021. [Online]. Available: https://towardsdatascience.com/a-brief-introduction-to-time-series-classification-algorithms-7b4284d31b97

[3] S. Aghabozorgi, A. S. Shirkhorshidi, and T. Y. Wah, "Time-series clustering–a decade review," *Information systems*, vol. 53, pp. 16–38, 2015.

[4] J. Brownlee, *Deep Learning for Time Series Forecasting: Predict the Future with MLPs, CNNs and LSTMs in Python*. Machine Learning Mastery, 2018. [Online]. Available: https://books.google.com/books?id=o5qnDwAAQBAJ

[5] E. Lewinson, "Introduction to hierarchical time series forecasting-part i," Mar 2021. [Online]. Available: https://towardsdatascience.com/introduction-to-hierarchical-time-series-forecasting-part-i-88a116f2e2

[6] R. J. Hyndman and G. Athanasopoulos, *Forecasting principles and practice*. Otexts, Online Open-Access Textbooks, 2021.

[7] P. Mancuso, V. Piccialli, and A. M. Sudoso, "A machine learning approach for forecasting hierarchical time series," *Expert Systems with Applications*, vol. 182, p. 115102, 2021.

[8] D. Burba and T. Chen, "A trainable reconciliation method for hierarchical time-series," *arXiv preprint arXiv:2101.01329*, 2021.